

端侧人工智能语言学习机

摘要

本作品是一款基于飞凌 ELF2（RK3588）学习板和 7 寸 MIPI 显示模块的人机交互英语学习终端。系统面向大学英语词汇和口语表达训练，将语音输入、端侧语音识别、大模型对话、语音合成、词汇状态管理和间隔复习整合到一台嵌入式设备中。用户可创建学习角色，选择科技、体育、文化、财经等感兴趣的方向，围绕近期新闻开展英语对话；系统会结合用户英语水平提炼新闻内容并生成讨论问题，使用户在了解时事的同时练习英语表达。系统在交流过程中自然引入目标词汇，识别用户表达中的重点词，提供即时释义，并把生词、会话和复习记录保存到本地数据库。

作品以 RK3588 为核心平台，利用多核处理器、神经网络加速单元和系统图形界面生态完成端侧人工智能部署。模型层已完成语音识别、15 亿参数量化大模型推理和英文语音合成的板端验证；其中语音识别中文实时倍率约 0.064、英文约 0.09，大模型生成速度约每秒 13.73 个词元，语音合成实时倍率约 0.18-0.21。应用层完成学习系统，包括角色管理、话题生成、聊天取词、生词本、复习卡片、间隔复习调度和多页面图形界面，并形成可替换的语音识别、大模型和语音合成接口。

与普通背单词软件相比，本作品的特点在于把词汇学习嵌入真实对话，把人工智能推理尽量放在设备端，把学习记录留在本地。系统既能作为嵌入式人工智能部署与人机交互演示平台，也能继续扩展为离线可用、隐私友好、具备长期学习记忆的专用英语学习机。

关键词：端侧人工智能；英语学习；时事新闻；语音交互；SM-2 间隔复习

第一部分 作品概述

1.1 功能与特性

本作品围绕“对话中学习”设计功能闭环。用户首先在首页创建或选择学习角色，填写英语水平和兴趣标签；系统根据角色信息生成适合的学习话题，用户进入聊天页后可通过键盘输入或语音输入与智能角色进行英语交流。智能回复时会结合当前话题、用户水平和目标词汇，尽量使用自然、简短、可模仿的表达。

话题内容以近期新闻为主要来源，用户可选择科技、体育、文化、财经等感兴趣的方向。系统对新闻内容进行筛选和提炼，并结合用户英语水平生成简短的背景介绍与讨论问题，使英语交流具有真实语境。用户既能接触近期时事，也能围绕感兴趣的内容练习词汇和口语表达。

在聊天过程中，系统支持点击单词查看释义，并可把生词加入个人词汇记录。生词保存后进入复习模块，用户可通过闪卡翻面、自评按钮和间隔复习算法进行间隔复习。系统同时保存用户角色、聊天会话、词汇状态、学习事件和复习结果，使一次聊天不只是即时问答，而能沉淀为持续学习记录。

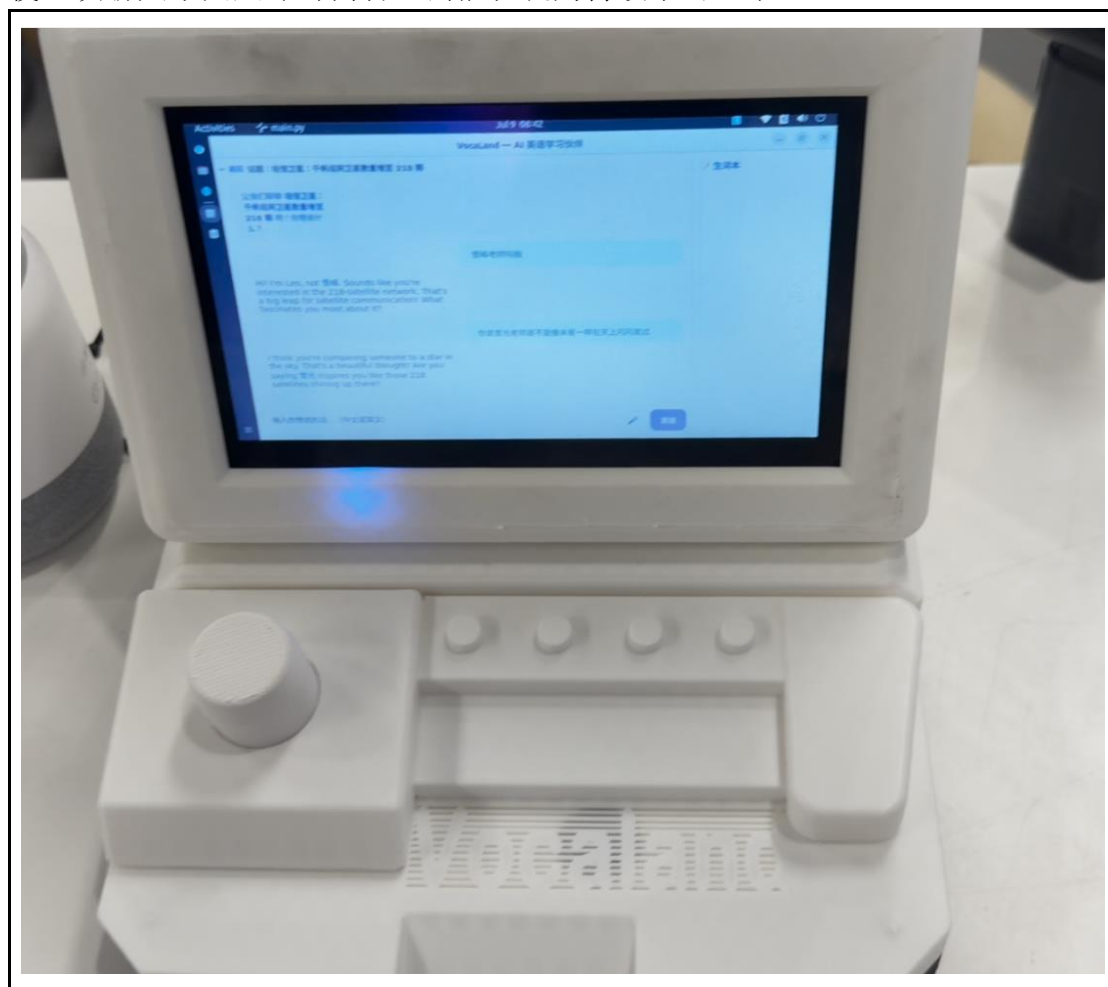


图 1-1 端侧人工智能语言学习机整机运行效果

系统支持模拟、云端和板端模型三种开发验证模式。模拟模式用于离线调试界面和学习逻辑，云端模式用于快速验证真实对话质量，板端模型模式用于最终展示端侧人工智能部署能力。

1.2 应用领域

本作品可用于大学英语、四六级词汇、口语启蒙和碎片化自学场景。与手机应用相比，专用设备形态更容易形成沉浸式学习环境，减少社交软件和娱乐应用干扰；本地数据库保存学习记录，也降低了长期学习数据上传到云端带来的隐私顾虑。

在教学和实验场景中，本作品可作为 RK3588 平台端侧人工智能综合实践案例，覆盖系统环境、音频外设、图形界面、本地数据库数据库、神经网络加速单元模型部署和多模块软件架构。它不仅展示一个学习应用，还能展示嵌入式人工智能产品从模型验证到应用集成的完整工程过程。

1.3 主要技术特点

系统采用“硬件平台+端侧模型+学习引擎+图形界面+本地数据”的分层结构。硬件层以 ELF2/RK3588 作为主控；模型层通过语音识别模型、大模型工具链和语音合成模型实现语音识别、对话生成和语音合成；学习层实现词汇状态机、间隔复习间隔复习和目标词追踪；界面层采用多页面导航；数据层通过本地数据库保存词汇库和用户学习过程。

模型接口采用适配器模式。语音识别、大模型和语音合成分别定义抽象基类，应用层只依赖统一接口，不直接绑定某个模型或推理程序。这样既便于电脑端用模拟/云端模式快速开发，也便于在 ELF2 上替换为语音识别模型、大模型工具链和语音合成模型等板端实现。

1.4 主要性能指标

指标	实测	效果
硬件平台	ELF2/RK3588, 8GB 内存, 32GB 存储, 7	运行稳定
语音识别	中文 RTF 约 0.064, 英文约 0.09	低延迟

指标	实测	效果
大模型推理	13.73 词元/s，加载约 7.3s	可端侧运行
语音合成	RTF 约 0.18-0.21	实时播报
应用测试	194 项通过	逻辑稳定

表 1-1 主要性能指标

1.5 主要创新点

1) 时事话题学习创新：将近期新闻按照科技、体育、文化、财经等兴趣方向转化为英语讨论内容，使用户在了解时事的同时练习英语表达，并在真实语境中完成取词和复习。

2) 端侧部署创新：在 RK3588 嵌入式平台上验证语音识别、大模型、语音合成三类智能模型，尽量减少对云端服务的依赖，提升隐私性和展示完整度。

3) 学习闭环创新：通过词汇状态机、学习事件和间隔复习调度，将单次对话转化为可持续跟踪的学习记录。

4) 工程架构创新：采用模型适配器、仓库仓库模式和图形界面框架页面分层，使硬件、模型、业务逻辑和界面能够并行开发、独立替换。

1.6 设计流程

作品设计流程分为需求分析、硬件验证、模型部署、应用开发、接口集成和性能测试六个阶段。需求分析阶段确定“端侧人工智能+英语学习机”的作品定位，明确离线性、专用设备形态和学习闭环为核心目标。硬件验证阶段完成 ELF2 开发板系统环境、音频设备和基础工具链检查。

模型部署阶段分别完成板端语音识别模型、大模型工具链大模型和英文语音合成的板端验证。应用开发阶段完成学习应用第二版学习系统，包括角色管理、话题生成、聊天页面、取词弹窗和复习页面。接口集成阶段将应用层抽象接口与板端语音识别适配器对接，并为大模型工具链和语音合成保留同样的接入方式。测试阶段通过单元测试、模型单项性能测试和语音录放测试验证系统可用性。



图 1-2 作品设计流程图

第二部分 系统组成及功能说明

本部分按照硬件系统、软件系统和功能模块说明作品组成。系统设计遵循先整体后局部的原则：先说明用户一次学习交互的数据流，再分别展开主控与外设、智能模型、学习引擎、图形界面和本地数据库。

2.1 整体介绍

系统由主控与显示模块、音频输入输出模块、模型推理模块、学习管理模块、交互界面模块和本地数据模块组成。用户说话后，音频由麦克风采集并送入语音识别模块，语音识别输出的文本进入对话模型；大模型根据角色设定、话题和目标词汇生成英文回复，回复文本一方面交给语音合成模块合成为语音，另一方面进入学习管理模块进行词汇扫描、状态更新和记录保存。

从系统边界看，硬件平台负责音频采集、显示输出和模型计算承载；模型推理模块负责理解和生成；学习管理模块负责把普通对话转化为可追踪的学习事件；图形界面负责把角色、话题、聊天和复习组织成连续体验；本地数据库数据库负责保存词库、角色、会话和学习记录。

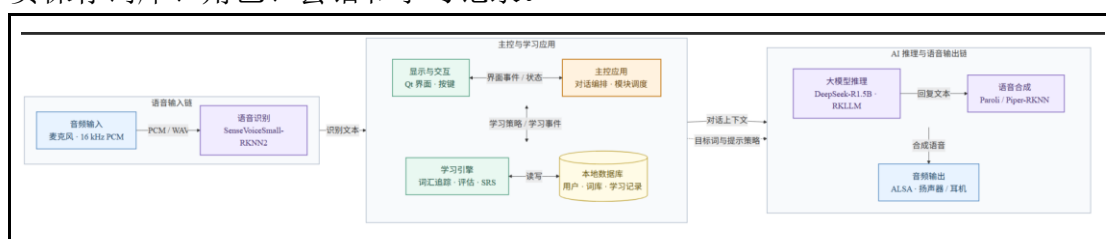


图 2-1 系统整体组成框图

该整体框图是后续模块说明的基础。硬件系统主要回答“设备由哪些部分组成”，软件系统主要回答“学习过程如何被程序组织”，性能参数部分则回答“各模块做到什么程度”。

一次完整学习交互可分为输入、理解、生成、反馈和沉淀五个步骤。输入阶段采集用户语音或文字；理解阶段将音频识别为文本，并结合角色信息判断用户意图；生成阶段由大模型给出英语回复；反馈阶段通过屏幕文本和语音合成语音返回给用户；沉淀阶段记录生词、会话和复习事件。

为了让学习过程更加自然，系统不要求用户每次先背单词再做题，而是在对话中暴露目标词汇。用户遇到不认识的词，可以直接点击气泡中的单词查看释义并加入生词本。该设计降低了单词学习的割裂感，使对话、查词和复习形成同一个学习循环。

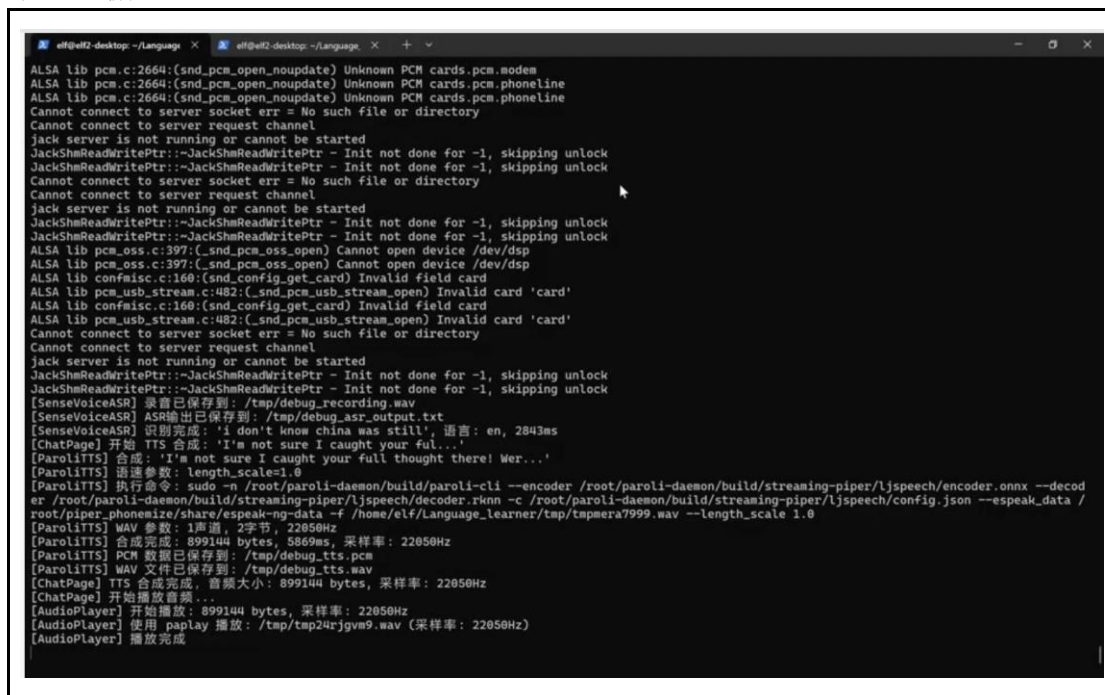


图 2-2 语音对话与学习记录数据流图

在当前实现中，语音识别已接入学习应用语音模式；大模型和语音合成已完成板端单项验证，应用层通过大模型接口和语音合成接口保留替换入口，后续可将云端大模型和模拟语音合成替换为大模型工具链和板端语音合成模块。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

硬件采用飞凌 ELF2 学习板作为主控平台，核心处理器为 RK3588。该芯片包含 4 个高性能核心和 4 个能效核心，板载 8GB 内存，具备约 6TOPS 的神经网络加速算力，适合在嵌入式端部署轻量级语音模型和大语言模型。

显示部分采用 7 寸显示接口液晶屏，分辨率为 1024×600，用于展示角色、话题、聊天气泡、生词面板和复习卡片。音频部分使用板载声卡、麦克风、3.5 毫米耳机接口或扬声器接口，实现录音输入和语音播放。调试阶段通过 USB、网口和命令行完成模型拷贝、日志查看和程序启动。

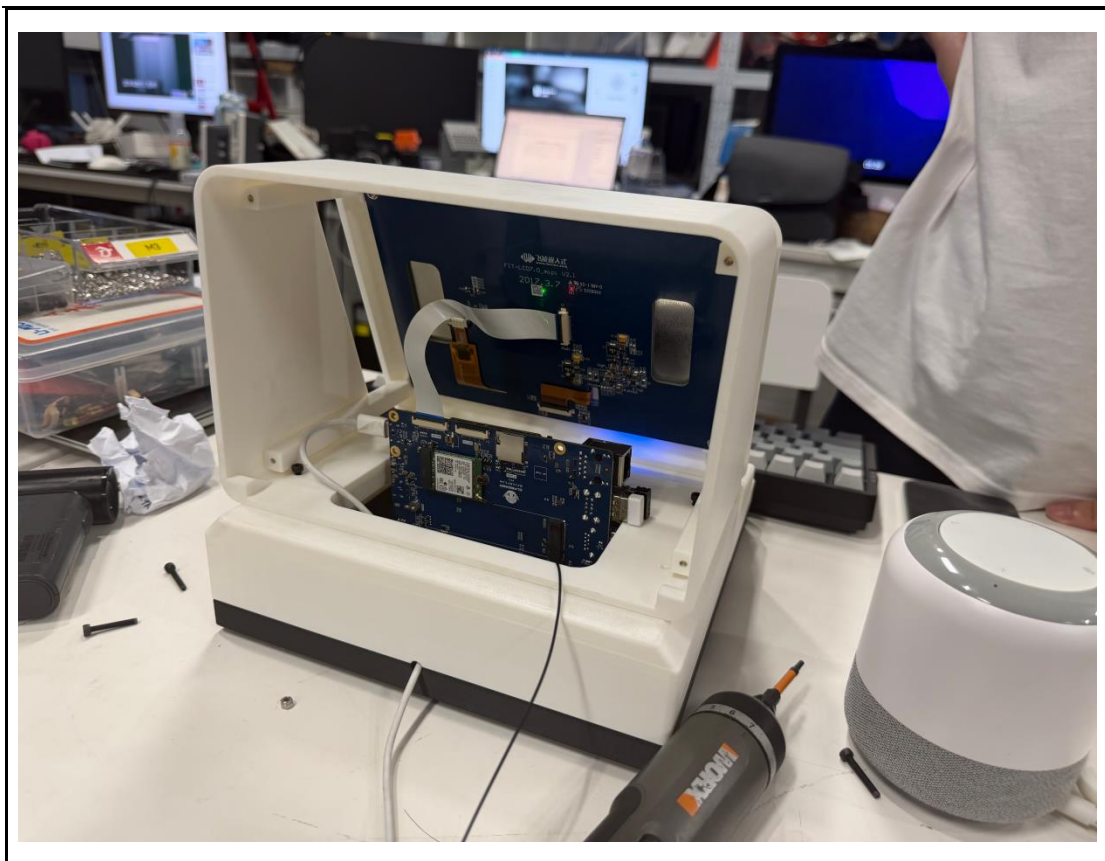


图 2-3 硬件整体组成示意图

模块	作用	状态
ELF2/RK3588	运行系统、界面和端侧模型	已验证
7 寸显示屏	显示学习界面和交互反馈	已接入
音频输入输出	录音、识别和语音播报	已验证
按键接口	录音触发、返回和确认	已预留

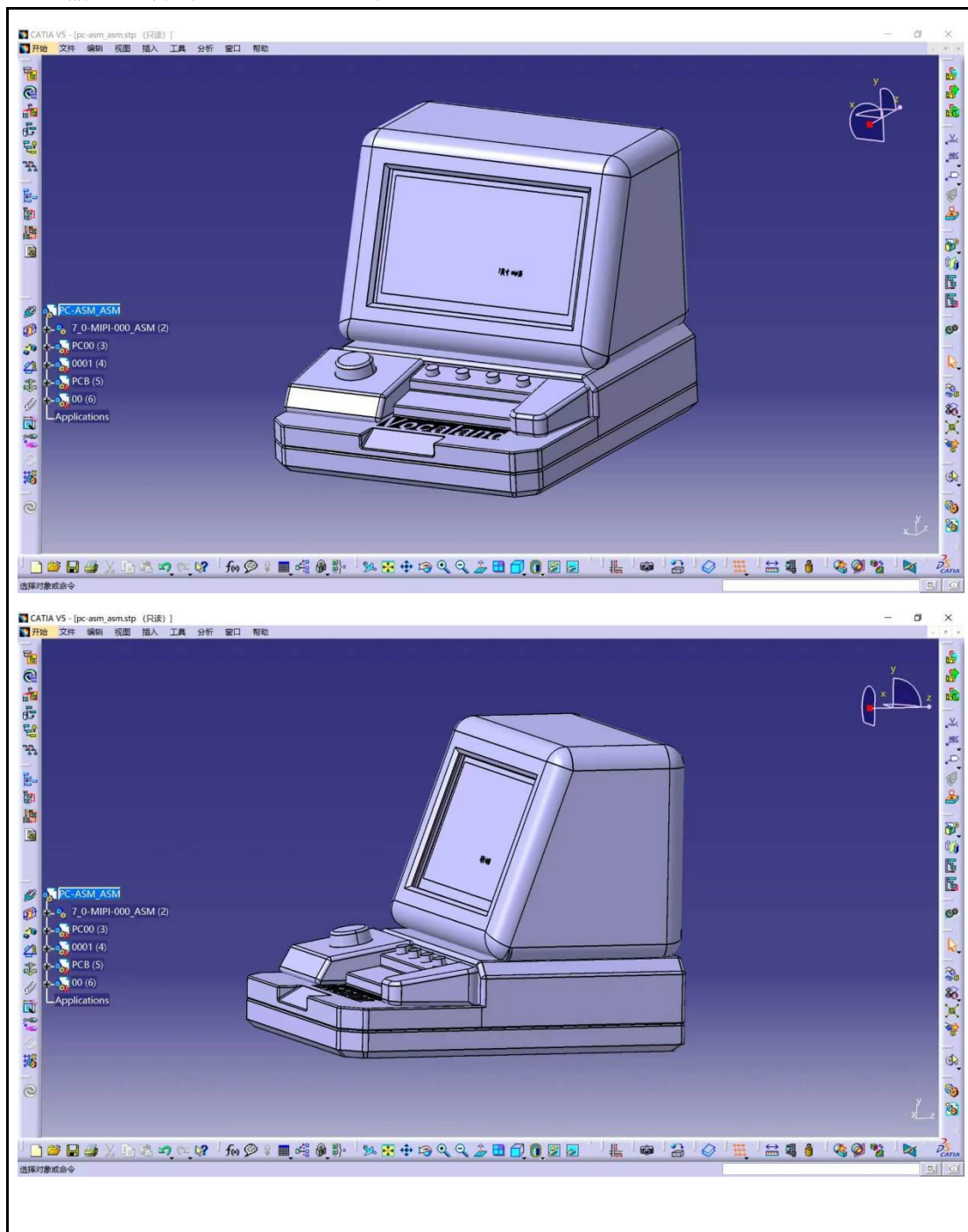
表 2-1 硬件模块说明

2.2.2 机械设计介绍

整机原型围绕“桌面或手持学习终端”的形态进行布置。7 寸屏作为主要交互面，主控板安装在屏幕后方或底部，音频接口、电源接口和调试接口布置在侧边或背部，便于接线和维护。由于 ELF2 开发板本身外设接口较多，原型阶段优先

保证散热、接口可达和装配可靠性。

机械结构设计预留了实体按键、扬声器开孔、散热空间和后续外壳安装位置。实体按键可承担录音触发、返回、确认等常用动作；扬声器开孔和耳机接口保证语音输出；背部空间用于主控板固定、电源走线和排线保护。



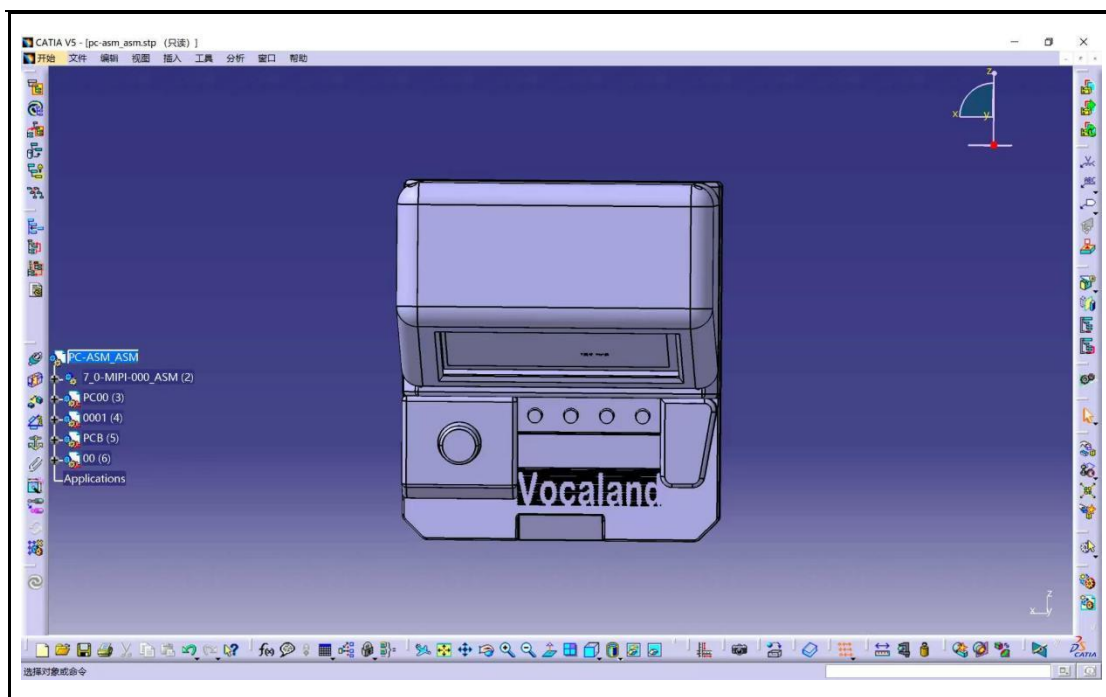


图 2-4 整机正面、侧面和背面结构视图

2.2.3 电路各模块介绍

电路系统以 ELF2 开发板板载资源为主体，外接显示、音频和调试外设。主控模块由 RK3588、内存、存储器和神经网络加速单元组成，负责系统运行、模型推理和界面渲染。显示模块通过显示接口连接 7 寸屏，承担学习界面输出。调试模块通过 USB 接口、网口和串口完成文件传输、运行日志查看和程序维护。

由于本作品重点是端侧人工智能学习系统，原型阶段没有额外设计复杂自制电脑 B，而是充分利用 ELF2 板卡成熟接口降低硬件调试风险。

主控板的神经网络加速单元资源用于语音识别、大模型推理和语音合成的加速验证。语音识别采用 RKNN 模型文件，大模型采用大模型工具链格式模型，语音合成中的解码器使用 RKNN 后端。三类模型的部署证明硬件平台能够承载端侧智能学习机的核心计算任务。

音频电路是语音学习机的关键输入输出链路。录音设备使用 ELF2 板载板载音频设备，测试时用录音和播放命令完成验证，并自动适配音频格式。实际调试中发现录音需要以立体声 16khz 方式采集，再交给语音识别模型处理，应用层因此在录音模块和板端语音识别模块适配器中保留立体声音频文件保存逻辑。

电源模块采用开发板标准 12v 直流输入，原型阶段以稳定供电和长时间调试为主要目标。按键接口模块作为后续实体按键接口预留，软件中已实现按键模块类，并在语音模式下支持通过按键接口或键盘快捷键触发录音。

音频调试结果表明，ELF2 板载音频链路可满足语音输入和播放需求。后续整机化时，可将耳机输出替换为小型扬声器，并用实体按键提升录音交互的确定性。

2.3 软件系统介绍

2.3.1 软件整体介绍

软件系统运行在系统桌面环境中，主体采用应用开发语言和图形界面框架实现。程序入口程序入口负责解析启动参数、初始化数据库仓库、创建大模型、语音合成和语音识别适配器，并把这些对象注入主窗口。主窗口采用页面栈组织页面，包含首页、话题页、聊天页和复习页，用户按学习流程在页面之间切换。

软件分层包括界面层、业务引擎层、模型适配层、数据仓库层和硬件适配层。界面层负责用户操作和视觉反馈；业务引擎层负责话题生成、词汇状态、复习调度和学习评估；模型适配层负责语音识别、大模型、语音合成的统一调用；数据仓库层通过仓库集中管理本地数据库读写；硬件适配层处理音频录放和按键接口触发。

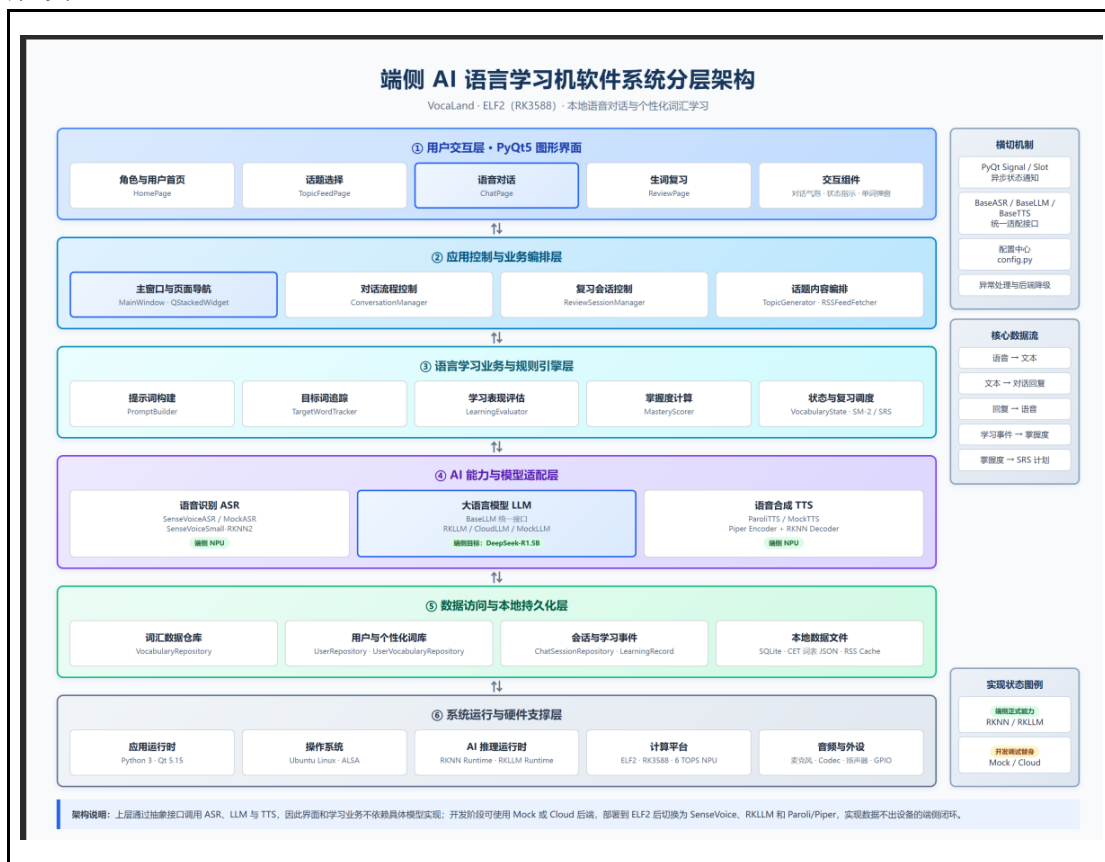


图 2-5 软件系统分层架构图

这种分层使模型部署线模型部署和应用开发线应用开发可以并行推进：应用

层先用模拟或云端接口完成交互和学习逻辑，模型层完成板端语音识别、大模型和语音合成验证后再逐步替换适配器。

2.3.2 软件各模块介绍

(1) 入口与界面模块。入口模块根据命令行参数选择模拟、云端或语音模式，并创建词汇仓库、用户仓库、会话仓库和用户词汇仓库等仓库对象。界面模块采用首页、话题页、聊天页、复习页四页面结构，首页用于角色选择，话题页用于生成或选择讨论主题，聊天页用于对话和取词，复习页用于闪卡复习。

首页支持创建和管理学习角色，角色信息包括姓名、英语水平和兴趣标签。话题页支持用户选择感兴趣的新闻领域，并根据近期新闻生成讨论话题，也保留自定义话题和本地兜底内容。聊天页采用气泡式对话布局，右侧保留本次会话生词面板；用户点击气泡中的英文单词时，系统通过取词弹窗显示释义并支持保存。复习页采用卡片翻转、自评按钮和完成统计，形成低门槛的复习体验。

新闻话题生成过程包括兴趣匹配、内容筛选、摘要提炼和问题生成。系统先根据用户选择的方向筛选近期新闻，再提取适合交流的事件背景，并按照用户英语水平生成英文话题、关键词和引导问题；当网络信息源不可用时，系统使用本地话题继续学习流程。

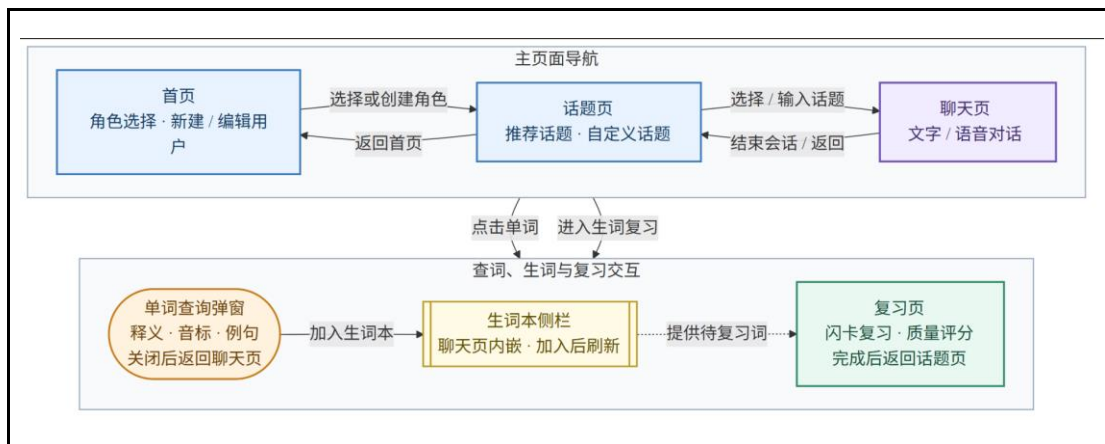


图 2-6 图形界面页面导航流程图

(2) 模型适配模块。语音识别、大模型和语音合成均采用抽象接口加具体适配器的形式。语音识别层包含模拟语音识别和板端语音识别模块，后者调用板端语音识别模型脚本并解析识别结果。大模型层包含模拟大模型和云端大模型，应用层通过大模型接口的聊天方法传入系统提示词和历史消息，后续可新增大模型工具链适配器。语音合成层包含模拟语音合成和板端语音合成模块，板端语音合成模块通过命令行调用语音合成命令生成音频文件并转换为音频数据数

据。

模型链路的设计重点是把“模型如何运行”与“学习流程如何组织”分开。应用层只关心输入文本、输出文本和延迟等结果，具体后端可以是云端接口、模拟对象或 RK3588 本地模型。这样既方便调试，也降低了后续更换模型的成本。

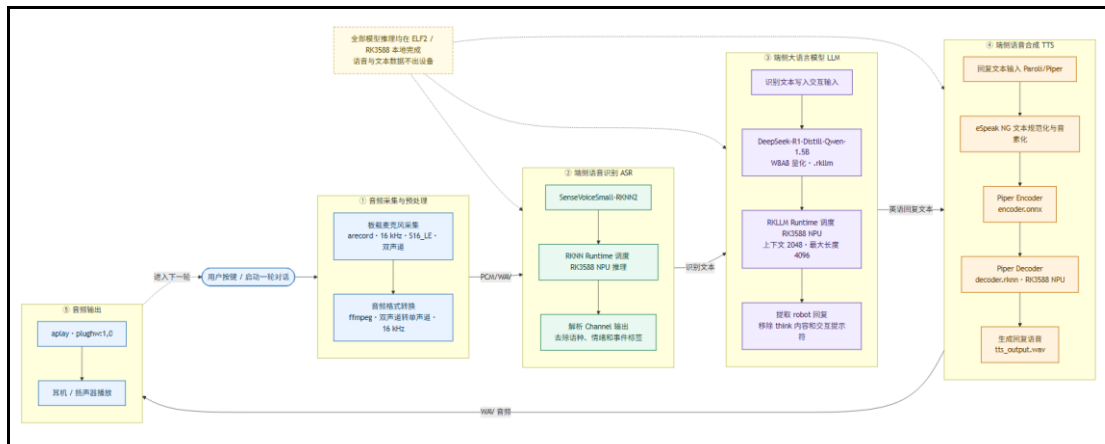


图 2-7 端侧模型推理流程图

板端模型部署结果显示，语音识别、大模型和语音合成均具备独立运行能力。其中语音识别已接入学习应用语音输入流程；大模型和语音合成在工程上已形成适配器或接入点，最终联调时可替换当前的云端大模型和模拟语音合成。

（3）学习管理模块。学习管理模块负责把对话文本转化为学习事件。系统会根据目标词、用户表达和智能回复更新词汇状态，并用学习事件记录被引入、被尝试、复习通过、复习失败等事件。掌握度评分模块根据用户表现调整掌握度分值，学习评价模块用于判断表达质量并产生轻量纠错反馈。

词汇状态机包含“未学习、已引入、已尝试、学习中、已掌握”五个阶段。系统在对话中自然引入一个词时，状态从“未学习”进入“已引入”；用户尝试使用该词后进入“已尝试”；当使用次数和质量达到要求后进入“学习中”，并接受间隔复习调度；连续多次高质量复习通过后进入“已掌握”。若用户使用错误或复习失败，状态可以回退。

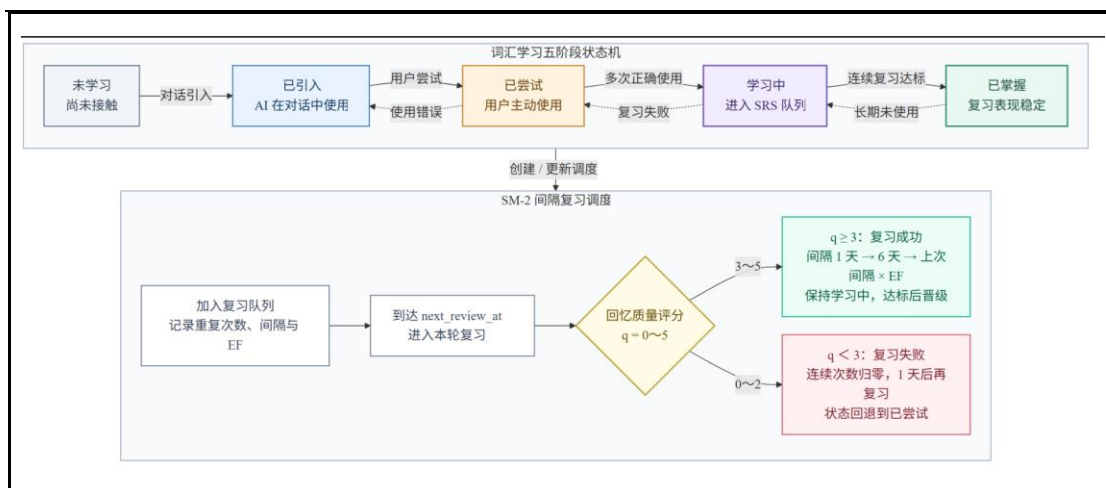


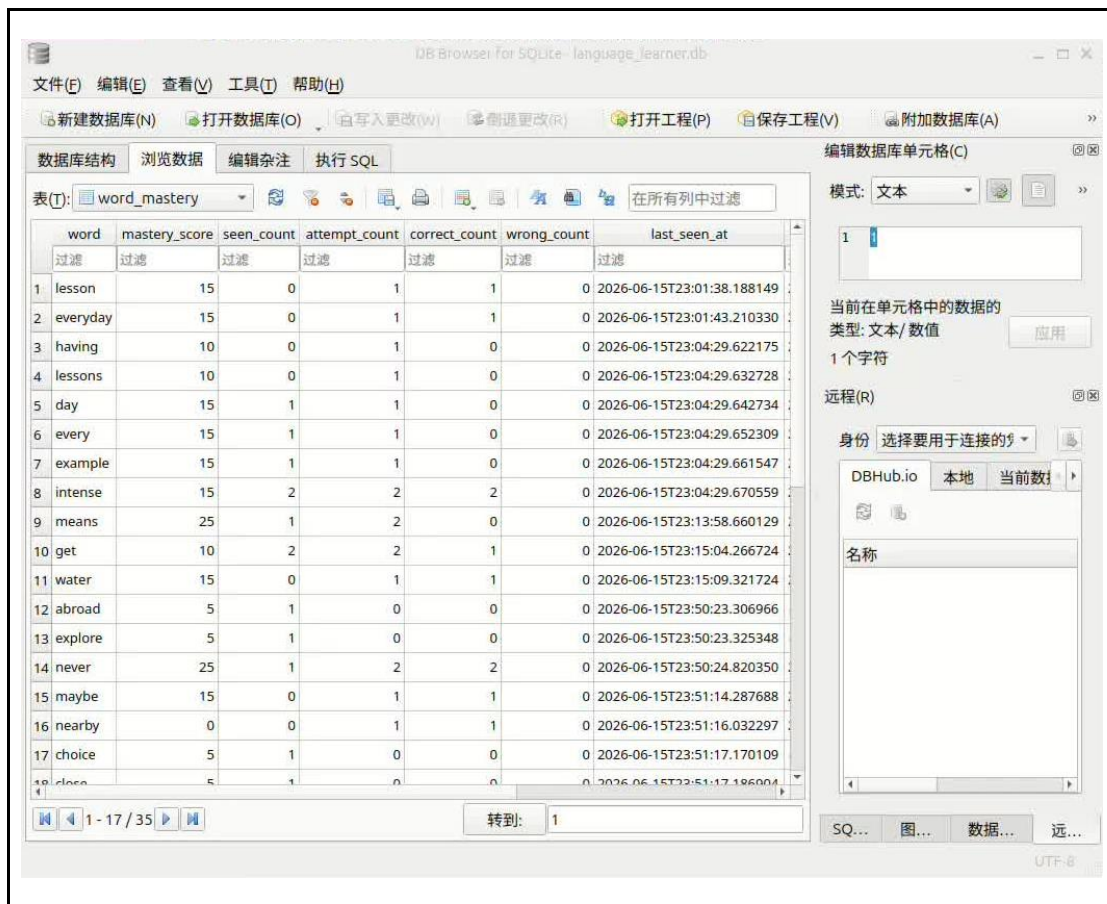
图 2-8 词汇状态机与复习调度流程图

复习模块采用 SM-2 间隔复习方法[6]，根据用户 0-5 分自评动态调整复习间隔。评分低于 3 分时回到 1 天；连续复习通过时，间隔按 1 天、6 天并结合难度因子逐步延长。通过固定评分序列的单元测试，可以验证间隔增长和失败重置是否正确。

（4）本地数据模块。系统使用本地数据库作为本地数据库，所有数据保存在单文件数据库中，不依赖额外数据库服务器，适合嵌入式设备部署。仓库仓库模式把数据库查询操作集中封装，界面和业务模块不直接拼写数据库查询语句，便于后续维护。

数据	主要内容	用途
词库	单词、音标、释义、例句、等级	保存基础词汇
掌握记录	掌握度、出现次数、最近评分	跟踪学习进度
学习事件	事件类型、复习质量、表达内容	记录学习过程
用户信息	姓名、水平、兴趣标签	保存个人偏好
会话记录	话题、开始时间、结束时间	保存学习历史
用户词汇	单词、状态、复习时间	管理个人生词

表 2-2 本地数据表设计



	word	mastery_score	seen_count	attempt_count	correct_count	wrong_count	last_seen_at
1	lesson	15	0	1	1	0	2026-06-15T23:01:38.188149
2	everyday	15	0	1	1	0	2026-06-15T23:01:43.210330
3	having	10	0	1	0	0	2026-06-15T23:04:29.622175
4	lessons	10	0	1	0	0	2026-06-15T23:04:29.632728
5	day	15	1	1	0	0	2026-06-15T23:04:29.642734
6	every	15	1	1	0	0	2026-06-15T23:04:29.652309
7	example	15	1	1	0	0	2026-06-15T23:04:29.661547
8	intense	15	2	2	2	0	2026-06-15T23:04:29.670559
9	means	25	1	2	0	0	2026-06-15T23:13:58.660129
10	get	10	2	2	1	0	2026-06-15T23:15:04.266724
11	water	15	0	1	1	0	2026-06-15T23:15:09.321724
12	abroad	5	1	0	0	0	2026-06-15T23:50:23.306966
13	explore	5	1	0	0	0	2026-06-15T23:50:23.325348
14	never	25	1	2	2	0	2026-06-15T23:50:24.820350
15	maybe	15	0	1	1	0	2026-06-15T23:51:14.287688
16	nearby	0	0	1	1	0	2026-06-15T23:51:16.032297
17	choice	5	1	0	0	0	2026-06-15T23:51:17.170109
18	close	5	1	0	0	0	2026-06-15T23:51:17.186004

图 2-9 本地数据库学习记录截图

当前词汇库设计包含 CET-4/CET-6 词汇、例句、短语、音标、难度和话题标签等字段。应用启动时会读取数据库词量并提示是否需要导入词库，便于调试阶段检查数据完整性。

（5）语音交互与异常处理模块。语音模式下，用户可以通过空格键、界面麦克风按钮或后续按键接口按键触发录音。录音模块采集音频后交给板端语音识别模块识别，识别文本返回聊天页作为用户输入。模型回复生成后，语音合成适配器可将文本转换为语音并通过播放模块播放。

在板端调试过程中，系统针对多个实际问题进行了处理：音频库默认设备不正确时指定输入设备输入设备 1；录音格式与语音识别模型测试环境不一致时保留立体声音频文件；语音识别模型将识别结果输出到错误输出时合并标准输出和错误输出解析；日志混入结果时只提取包含通道标记的识别行。这些处理提高了真机语音输入的稳定性。

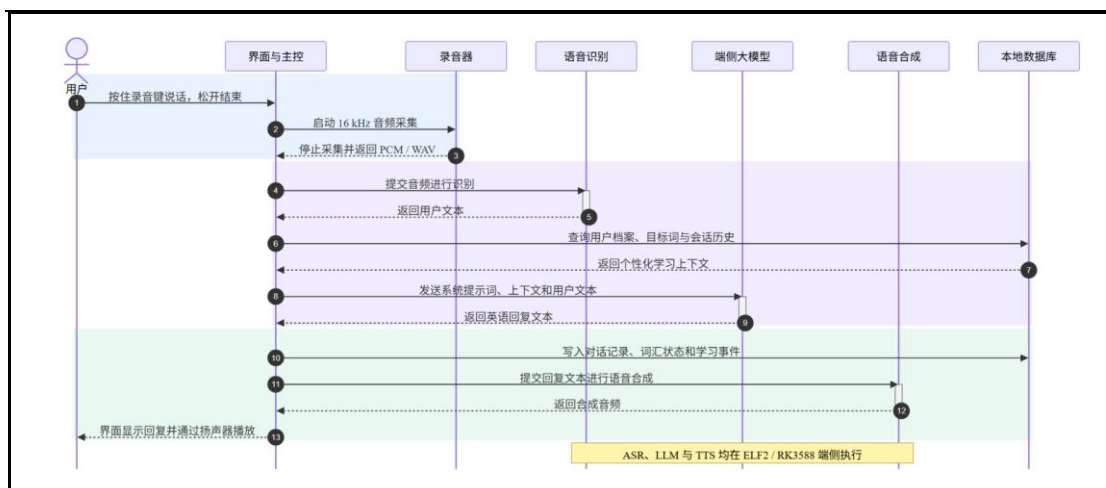


图 2-10 一次语音对话的时序流程图

异常处理方面，语音识别、大模型、语音合成适配器均提供可用性检查或空结果返回机制。模型不可用时，系统可以退回模拟或云端模式继续验证界面和学习逻辑，避免单个模型环节阻塞整个应用开发。

第三部分 完成情况及性能参数

本作品当前已完成硬件平台环境验证、三类智能模型板端部署验证、学习应用应用层开发、本地数据库数据层设计以及语音识别到应用的语音输入集成。最终整机展示可围绕“设备启动、选择角色、选择话题、语音或文字对话、点击取词、保存生词、闪卡复习”这一流程进行。

3.1 整体介绍

整机原型以 ELF2 主控板和 7 寸显示屏为核心，软件启动后进入学习应用图形界面。用户在首页选择角色后进入话题页，系统根据兴趣标签生成话题卡片；进入聊天页后，用户可输入英语或中文内容，人工智能以英语进行回复；在聊天过程中点击单词可以查看释义并保存，之后可进入复习页进行卡片复习。



图 3-1 整机正面运行效果



图 3-2 整机斜 45 度运行效果

3.2 工程成果

3.2.1 机械成果

机械方面，作品形成了以开发板和显示屏为主体的功能样机布局。屏幕作为主要交互面，主控板承担计算和接口连接，音频输入输出和调试接口保持可访问。当前结构更偏工程原型，适合持续调试和竞赛展示；后续可在此基础上完成外壳、

按键、扬声器和电池的一体化设计。

结构布局时重点考虑三点：第一，屏幕和主控板固定关系清晰，便于评委观察作品形态；第二，MIPI 排线、电源线和音频线保持足够弯折空间，避免频繁插拔造成损伤；第三，散热和接口区域不被封死，保证长时间模型测试和现场演示稳定。

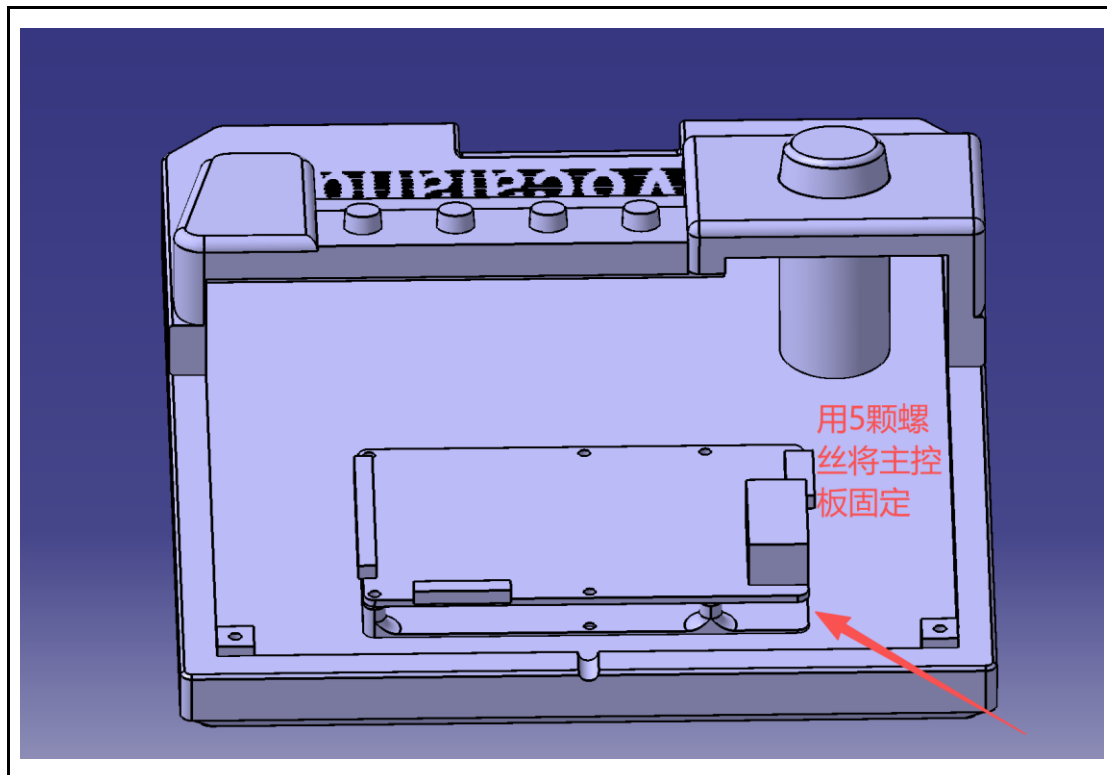


图 3-3 主控板与壳体装配关系



图 3-4 调试接口与外设连接状态

3.2.2 电路成果

电路与接口方面，完成了 ELF2 主控板、音频输入输出和调试链路验证。音频设备识别为板载音频芯片，录音和播放均通过系统命令完成。语音识别测试中，录音以 16 千赫兹立体声格式采集，并可被板端语音识别模型正确识别。

模型部署方面，板端语音识别模型使用 RKNN 文件进行语音识别；15 亿参

数量化大模型使用大模型运行工具加载量化模型；语音合成模型使用编码器和解码器完成英文语音合成。三类模型的独立运行证明硬件平台具备端侧智能学习机的基础计算能力。

3.2.3 软件成果

软件成果包括学习应用界面、学习业务引擎和模型适配接口。首页提供角色创建和选择功能，用户可以根据英语水平和兴趣方向建立不同学习角色。话题页提供话题卡片、自定义话题和复习入口，使学习内容与用户兴趣关联。

聊天页采用双栏布局，左侧显示对话气泡和输入框，右侧显示本次会话生词。系统支持点击气泡中的单词弹出释义窗口，并可把单词加入个人生词记录。复习页采用闪卡翻转和再练/较难/掌握等自评按钮，使用户能够对保存的生词进行快速复习。

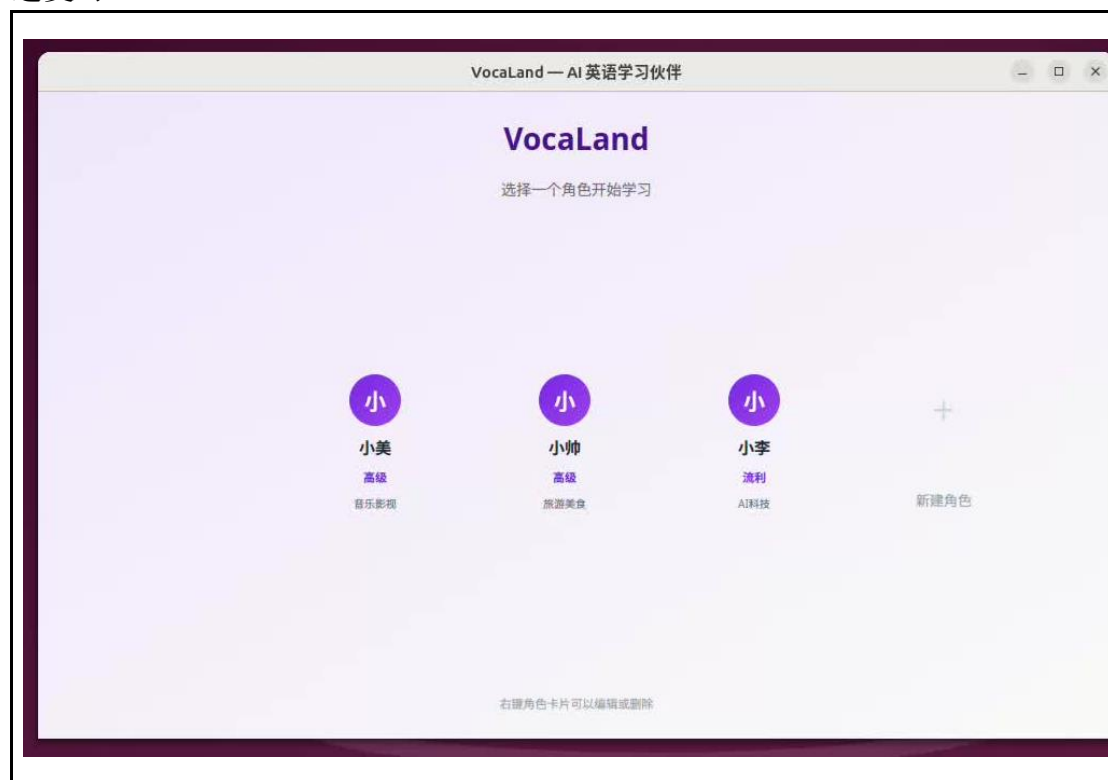


图 3-5 首页与角色创建界面

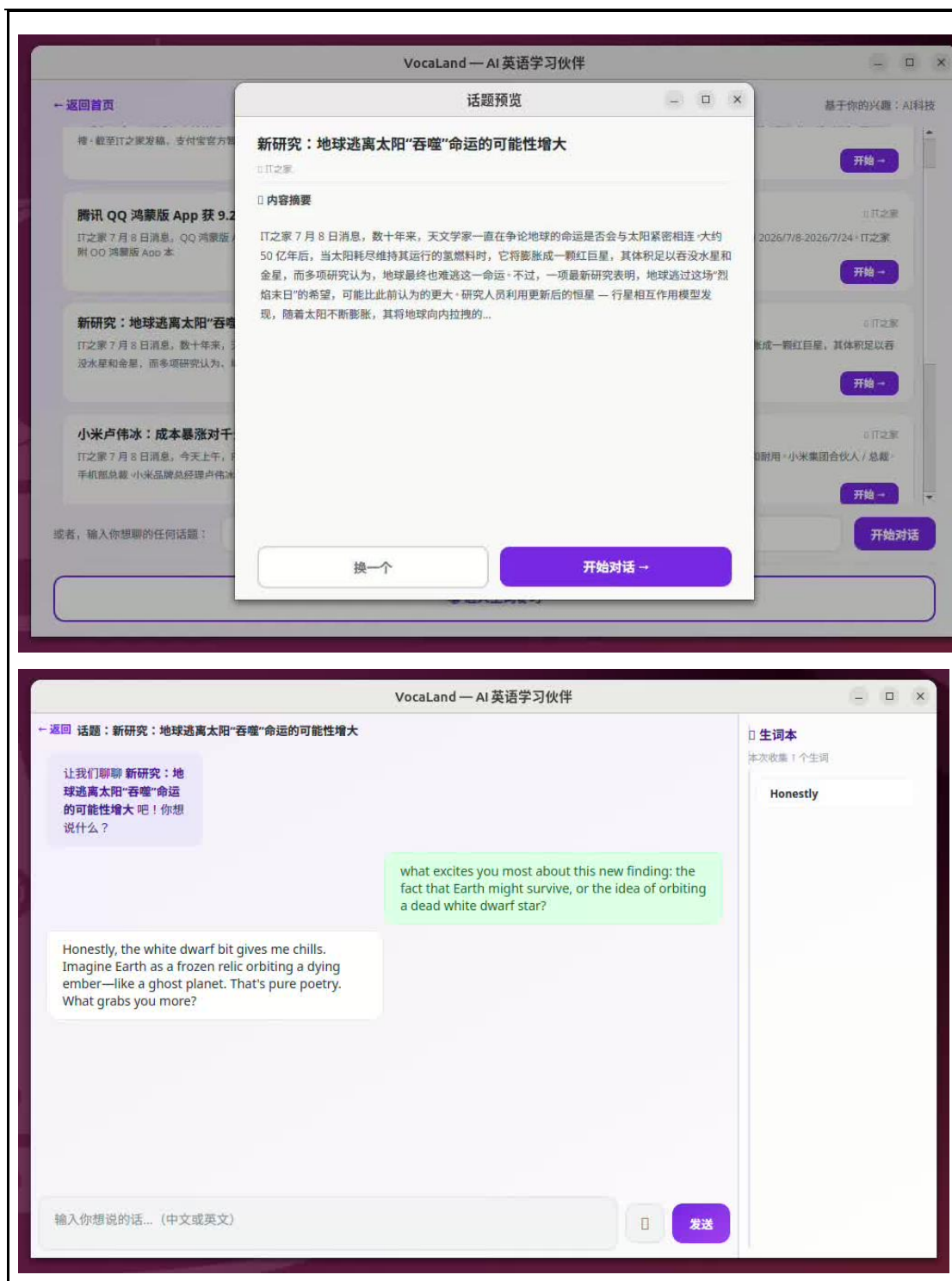


图 3-6 话题选择与聊天界面

应用层通过 194 个单元测试覆盖仓库、状态机、话题生成、复习管理、语音识别/语音合成接口等模块，说明核心逻辑具有较好的可验证性。

软件成果的第二个重点是学习闭环。系统不只显示智能回复，还把聊天中的词汇转化为学习对象。取词弹窗取词弹窗优先从本地词库查询释义，必要时可调用大模型生成兜底解释；用户词汇仓库保存用户词汇状态，复习管理模块根据用

户自评更新复习结果。

复习页面采用正反面卡片结构，正面显示英文词，翻转后显示中文释义和例句，用户根据掌握情况选择再练、较难或掌握。系统利用间隔复习算法计算下次复习间隔，使容易词减少重复、困难词增加复现。

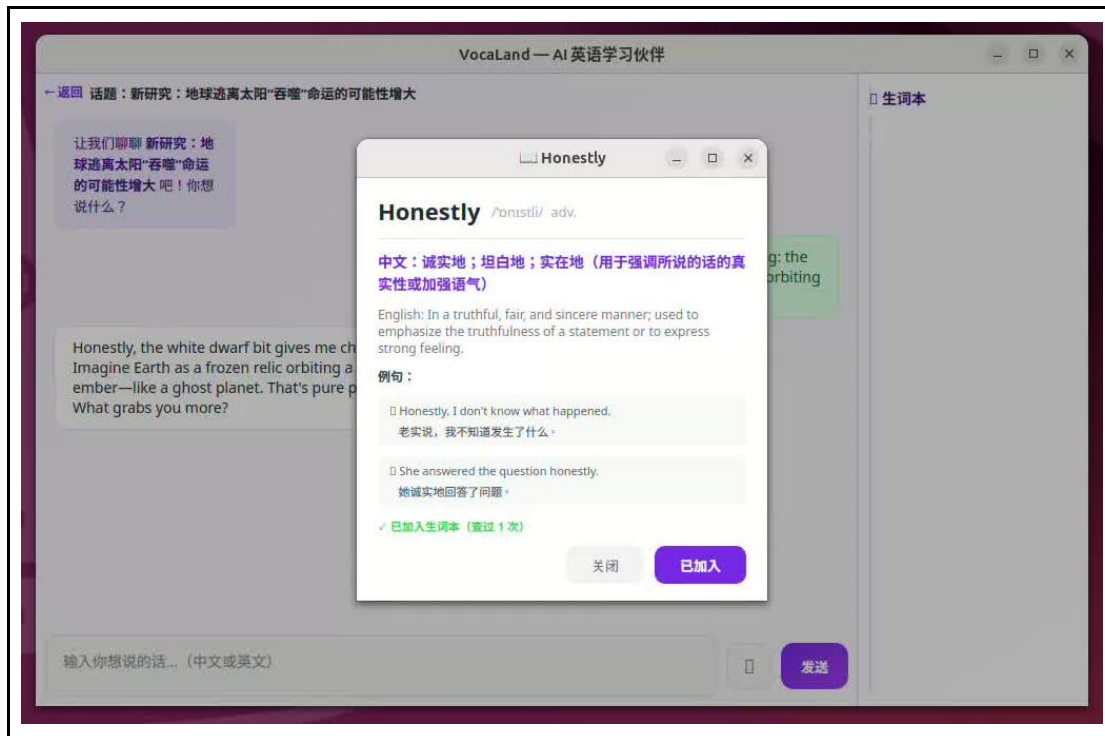


图 3-7 即时查词与生词保存界面

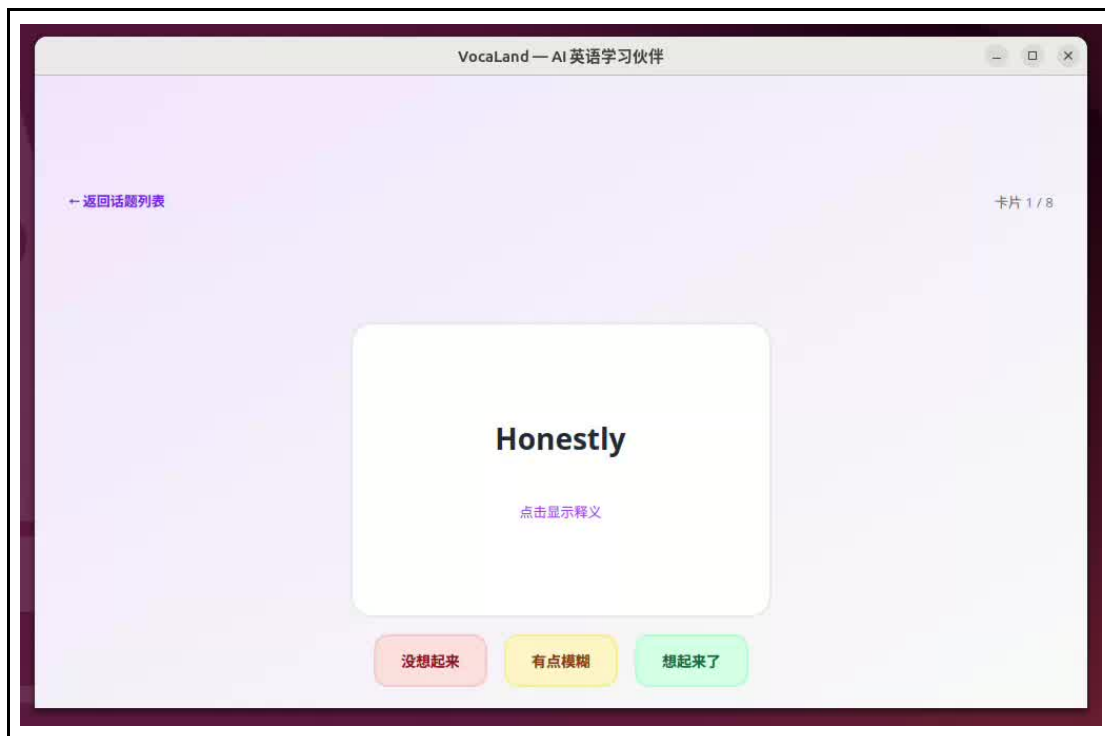


图 3-8 生词本与复习卡片界面

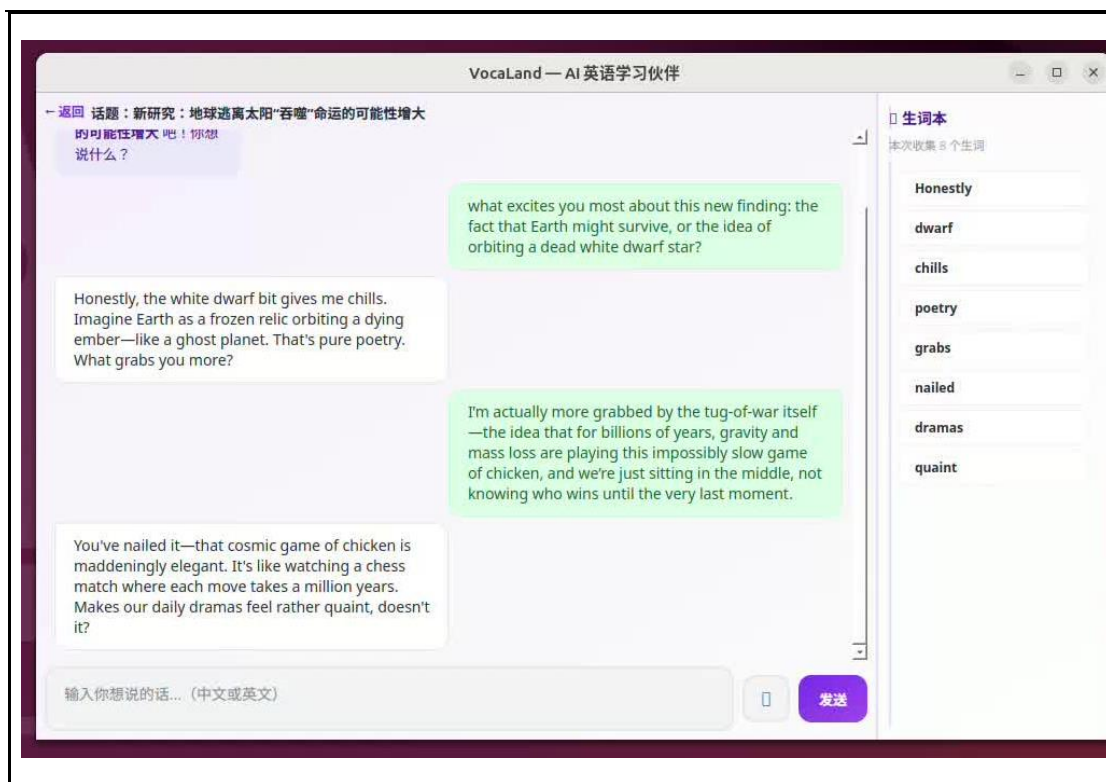


图 3-9 学习记录或数据库保存结果

3.3 特性成果

(1) 语音交互成果：板端语音识别模型已完成板端部署，并接入学习应用语音模式。系统可通过音频库录音、保存立体声音频文件、调用语音识别模型脚本、解析识别结果并把文本送入聊天流程。中文识别测试实时倍率约 0.064，英文识别测试实时倍率约 0.09。

(2) 端侧推理成果：15 亿参数量化大模型已在大模型运行工具下加载运行，生成速度约每秒约 13.73 个词元，内存占用约 1.73GB，模型加载约 7.3 秒。该结果说明 15 亿参数量化语言模型能够在 RK3588 平台上完成可用的文本生成。

(3) 语音合成成果：板端英文语音合成模型英文语音合成已完成部署，合成实时倍率约 0.18-0.21，约 4.8-5.5 倍实时。由于学习机主要输出英语，当前英文语音合成可以满足英语回复播报需求。

(4) 时事话题成果：系统可按照用户选择的兴趣方向生成近期新闻话题和英语引导问题，使时事了解、口语交流和词汇学习在同一次对话中完成。

(5) 学习闭环成果：应用层完成角色、话题、对话、取词、保存和复习功能，形成从输入到学习记录的完整闭环；复习模块按照 SM-2 规则安排下次复习时间，并通过单元测试验证主要调度逻辑。



```
@dataclass
class ASRResponse:
    """ASR 的一次转写结果。
    dataclass 自动生成 __init__，你只需要声明字段和类型。
    """
    text: str # 转写文本
    latency_ms: float = 0.0 # 识别耗时（毫秒）
    language: str = "auto" # 检测到的语言（zh/en/auto）
    confidence: float = 0.0 # 置信度 0.0-1.0

class BaseASR(ABC):
    """ASR 抽象基类—所有语音识别后端的统一接口。
    子类必须实现：
    transcribe(): 音频字节流 → 转写文本
    is_available(): 检查后端是否可用
    """
    @abstractmethod
    def transcribe(self, audio_bytes: bytes) -> ASRResponse:
        """将音频字节流转写为文本。
        参数：
        audio_bytes: 原始 PCM 音频数据(16kHz, 16bit, mono)
        返回：
        ASRResponse: 包含转写文本和元信息
        """
    @abstractmethod
    def is_available(self) -> bool:
        """检测 ASR 后端是否可用。
        MockASR 永远返回 True，
        SenseVoiceASR 会检测模型文件是否存在、NPU 是否就绪。
        """
    @property
    def name(self) -> str:
        """后端名称，方便日志输出。"""
        return self.__class__.__name__
```

定义了 ASR 的统一接口，所有 ASR 都要实现这个方法



```
def _save_as_wav(self, audio_bytes: bytes) -> str:
    """将 PCM 数据保存为 WAV 文件。保留立体声格式(SenseVoice 支持)。"""
    wav_path = tempfile.mktemp(suffix=".wav")
    # 直接保存立体声 (AudioRecorder 输出的就是立体声)
    # SenseVoice 会自动处理立体声输入
    with wave.open(wav_path, "wb") as wf:
        wf.setnchannels(2) # 立体声
        wf.setsampwidth(2) # 16-bit = 2 bytes
        wf.setframerate(16000) # 16kHz
        wf.writeframes(audio_bytes)
```

原版直接保存立体声，没有转单声道，这是识别率低的根源。我们将录制的音频转成双通道。

```
def start(self):
    """开始录音。启动后台线程，立即返回。"""
    if self._recording:
        return

    self._frames = []
    self._recording = True
    self.recording_started.emit()

    self._thread = threading.Thread(target=self._record_thread, daemon=True)
    self._thread.start()
```

打开指定的录音设备，input_device_index=1 指定 ELF2 的声卡。

```
while self._recording:
    try:
        data = stream.read(self.CHUNK, exception_on_overflow=False)
    except Exception:
        break
    self._frames.append(data)

    # 计算 RMS 电平
    sample_count = len(data) // 2
    if sample_count > 0:
        samples = struct.unpack(f"<{sample_count}h", data)
        rms = math.sqrt(sum(s * s for s in samples) / sample_count)
        level = min(rms / 32768.0 * 100.0, 100.0)
        self.audio_level.emit(level)
```

持续录音，直到 _recording 变为 False



```
# 线程退出时返回录音数据
all_data = b"".join(self._frames)
self._frames = []
self._recording = False
self.recording_finished.emit(all_data)
```

录音结束后，把所有音频帧合并成一个完整的 PCM 数据，通过信号发送出去

```
def _on_recording_finished(self, pcm_data: bytes):
    """录音完成 → 后台线程跑 ASR 转写。"""
    if not pcm_data or len(pcm_data) < 3200: # < 0.1 秒 = 误触
        self._reset_input_state("输入你想说的话... (中文或英文)")
        return

    def _run():
        try:
            result = self._asr.transcribe(pcm_data)
            text = result.text
        except Exception:
            text = ""
        # pyqtSignal 自动跨线程排队到主线程
        self.asr_result_ready.emit(text)

    threading.Thread(target=_run, daemon=True).start()
```

录音结束后，启动后台线程调用 ASR 识别语音

```
def _on_asr_result(self, text: str):
    """ASR 结果回到主线程 → 填入输入框 → 自动发送。"""
    self._reset_input_state("输入你想说的话... (中文或英文)")

    if text and text.strip():
        self.input_box.setPlainText(text.strip())
        # 短暂延迟让用户看到识别结果，然后自动发送
        QTimer.singleShot(200, self._on_send)
```

收到 ASR 结果后，显示到输入框，然后自动发送

图 3-10 语音交互与端侧推理测试过程

性能测试采用分模块验证方式。语音识别、大模型和语音合成均在 ELF2/RK3588 平台完成单项测试；应用层通过单元测试验证业务逻辑。由于最终

语音识别→大模型→语音合成连续链路仍需统一联调，端到端平均响应时间将在最终版本中补充。

测试项目	方法	实测	结论
中文识别	固定中文音频输入	RTF 约 0.064	满足实时识别
英文识别	固定英文音频输入	RTF 约 0.09	满足口语输入
大模型推理	加载 15 亿参数量化模型	13.73 词元/s，内存约 1.73GB	可端侧生成
语音合成	生成英文语音	RTF 约 0.18-0.21	满足英文播报
应用测试	全量单元测试	194 项通过	核心逻辑稳定
端到端延迟	录音到播放全链路	待最终联调补测	后续优化指标

表 3-1 性能测试结果

第四部分 总结

4.1 可扩展之处

后续扩展可以按照“先打通核心链路、再完善交互体验、最后沉淀学习内容”的思路推进。第一，端侧语音闭环的完善应采用接口分层和逐段联调的方法：先固定语音识别、大模型和语音合成三个模块的输入输出格式，再用日志记录每一段的耗时、文本结果和异常信息；随后将当前云端大模型和模拟语音合成逐步替换为板端适配器，形成“录音—识别—生成—合成—播放”的完整闭环。这样可以在不大改应用层代码的情况下，逐步完成模型工具链和学习应用的最终接入。

第二，独立学习机形态可以采用“高频操作实体化、复杂操作触屏化”的方法扩展。实体按键主要承担开始录音、停止录音、返回和确认等高频动作，触屏继续负责角色选择、话题选择、取词和复习操作。实现时可先抽象统一的输入事件层，把键盘、触屏和实体按键都转换为同一类操作事件，再由界面层响应。这样既能保留现有软件结构，也方便后续更换外设。

第三，学习效果量化可以按照“采集数据—形成指标—生成反馈”的方法实现。系统在对话、取词和复习过程中记录发音文本、目标词使用情况、复习结

果和错误类型；发音评分可从音素相似度、完整度和流利度三个维度给出分数，逐句纠错则重点提示语法错误、用词不当和更自然的表达。学习报告页面可以按日或按周汇总学习时长、新增词汇、掌握词汇、复习正确率和常见错误，使学习进步更直观。

第四，整机完成度需要采用产品工程迭代的方法提升。外壳设计可先根据开发板、屏幕、电池、扬声器和散热片尺寸建立装配空间，再通过简单样件验证开孔位置、握持角度和屏幕可视角度；电池与扬声器部分重点验证续航、音量和发热情况；散热设计则通过连续语音交互和大模型生成测试观察温度变化，再决定是否增加导热片、散热孔或风道。这样可以把外观优化和稳定性验证结合起来。

第五，内容体系扩展应采用分层课程和持续更新的方法。词库可以按中考、高考、CET-4/CET-6 和日常口语分级，话题库按新闻、校园、科技、生活、文化等方向分类，并为每类话题配置不同难度的引导问题。课程层面可设计“听新闻—聊观点—取生词—复习巩固”的固定流程，再结合 SM-2 复习结果动态推荐下一次练习内容，使作品从一次演示样机逐步走向可长期使用学习工具。

4.2 心得体会

本作品的开发过程使我们认识到，嵌入式人工智能产品不是把模型简单搬到开发板上，而是要同时处理硬件资源、模型性能、交互体验和学习逻辑之间的平衡。最初设想是一台“会聊天的背单词机”，但实际推进中发现，真正影响体验的不只是模型能否回答问题，还包括录音设备是否稳定、识别结果如何传入界面、回复是否适合学习者水平、点击查词是否顺手、复习记录能否持续保存等细节。

在模型部署方面，RK3588 平台给了作品较强的端侧计算基础，但也带来了内存、编译、权限和音频格式等工程问题。例如语音框架编译时需要控制并行数避免内存不足，语音识别模型输出需要解析错误输出，录音格式需要与模型测试环境保持一致，语音合成运行还涉及依赖库路径和权限。这些问题让我们更加理解“能运行”和“能稳定集成到产品中”之间的差距。

在应用层设计方面，学习应用从单纯聊天界面逐步扩展为角色、话题、聊天、取词和复习组成的学习系统。我们体会到，学习机的价值不在于一次回答多么复杂，而在于能否把每次对话变成可积累的学习记录。后续继续完善端到端联调和整机外观后，本作品可以更完整地展示端侧人工智能在教育终端中的应用潜力。

第五部分 参考文献

[1] FORLINX EMBEDDED. User's Hardware Manual V1.5[EB/OL]. [2026-07-09].

http://docs.forlinx.net/rockchip/ok3588-c/OK3588-C_User_Hardware_Manual.html.

[2] ROCKCHIP. rknn-llm: RKLLM Runtime and Toolkit[CP/OL]. [2026-07-09].

<https://github.com/airockchip/rknn-llm>.

- [3] K2-FSA. sherpa-onnx: Speech-to-text, Text-to-speech, and Other Speech Processing with Next-gen Kaldi[CP/OL]. [2026-07-09].
<https://github.com/k2-fsa/sherpa-onnx>.
- [4] SQLITE CONSORTIUM. SQLite Documentation[EB/OL]. [2026-07-09].
<https://sqlite.org/docs.html>.
- [5] THE QT COMPANY. Qt for Python[EB/OL]. [2026-07-09].
<https://doc.qt.io/qtforpython-6/>.
- [6] WOZNIAK P A. SuperMemo 2: Algorithm[EB/OL]. [2026-07-09].
<https://super-memory.com/english/ol/sm2.htm>.
- [7] THOMASTHEMAKER. SenseVoiceSmall-RKNN2[CP/OL]. [2026-07-09].
<https://huggingface.co/ThomasTheMaker/SenseVoiceSmall-RKNN2>.
- [8] MARTY1885. Paroli: Streaming TTS Based on Piper with Optional RK3588 NPU Support[CP/OL]. [2026-07-09].
<https://github.com/marty1885/paroli>.